



DoubleVariable

🕒 Created time	June 20, 2026 1:06 PM
☰ Category	CommonVariables
👤 Last edited by	 R.A R.A
🕒 Last updated time	July 31, 2026 5:49 AM

```
//Doubles can have a Decimal point - 1.23, 43

double averageAge;

//averageAge = (43.0 + 22 + 62) / 3;

averageAge = 2.3 / 3;

Console.WriteLine(averageAge);
```

1. היכרות עם הטיפוס Double

למשתנים מסוג `double` יכולה להיות נקודה עשרונית – למשל 1.23, או 43. הטיפוס `double` קצת שונה מ-`int` מכיוון שהוא מאפשר שמירת נקודה עשרונית. אם נסתכל על המספר 43, אפשר לשאול: למה שיישמר כ-`double` אם אין לו נקודה עשרונית?

התשובה היא: מבחינה טכנית, המספר יכול להכיל נקודה עשרונית (43.0). כך או כך זה בסדר – משתנה `double` לא חייב להכיל בפועל ערך עשרוני והוא אינו נעול על כך, אך האפשרות הזו תמיד קיימת.

2. ה-Type Inference בחישובים

התוצאה של הקוד הזה תחושב כ-Int (מספר שלם):

```
averageAge = (43 + 21 + 62) / 3;
```

במקרה הזה, בגלל שהקומפיילר מזהה רק מספרים שלמים, הוא מחשב את התוצאה כ-`int`.

זה אומר שבמהלך פעולת החילוק נאבד את השארית העשרונית שלנו, ורק התוצאה השלמה תיכנס לתוך `averageAge` גם אם הוא מוגדר כ-`double`. אבל אם נכתוב את אותו קוד כך:

```
averageAge = (43.0 + 21 + 62) / 3;
```

עכשיו התוצאה תהיה

```
double = 42.333333333333336.
```

כאשר מספר כתוב כשלם, הקומפיילר מתייחס אליו כ-`int`.
ברגע שאנחנו מוסיפים `0.` למספר למשל `43.0` - שהוא עדיין אותו מספר, אנחנו נותנים לקומפיילר רמז.
הקומפיילר מזהה את הטיפוס כ-`double`, מתייחס גם לשאר המספרים בתרגיל כ-`double`, ושומר על רמת הדיוק העשרונית בתוצאה הסופית.

3. ה-Casting בין Int ל-Double

שימו לב שערך מטיפוס `int` יכול להיכנס לתוך משתנה מטיפוס `double` ללא שום בעיה. Implicit Casting

אנחנו עוברים ממשנה פחות ספציפי למשנה יותר ספציפי.

לכן, המספר 43 יכול להיות `int` ולהיכנס למשתנה `double` מבלי לשנות דבר - כי 43 זה פשוט 43.

אבל אם היינו מנסים את הכיוון ההפוך - להכניס `double` לתוך `int` למשל את המספר 1.23 - המערכת הייתה חייבת לחתוך את ה-.23. מהלך כזה יוביל ל-Data Loss, ולכן הקומפיילר לא יאפשר זאת.

אי אפשר להכניס `double` ל-`int` גם אם הערך הוא שלם לחלוטין כמו 43.0, כי המערכת חייבת להבטיח בטיחות ולא מניחה שזה תמיד יהיה המצב.

4. פעולות מתמטיות Math in .NET

```
averageAge = 2.3 / 3;
```

היתרון הגדול של טיפוס `double` הוא היכולת שלהם להחזיק נקודה עשרונית, וזו הסיבה שהם הטיפוס המספרי הנפוץ ביותר בכל הקשור למתמטיקה.

בעזרת `double` אפשר **לחבר**, **לחסר**, **להכפיל**, **לחלק**, **להוציא שורש ריבועי** – בעצם כל פעולה מתמטית.

אם תכתבו ב-C# את הפקודה `Math.`, תפתח הספרייה המתמטית של **.NET**. כל הפונקציות שם עובדות עם `double`, מכיוון שזהו הטיפוס הסטנדרטי למספרים עשרוניים.

יעילות ודיוק:

הטיפוס `double` נחשב לזול ויעיל יותר מבחינת משאבי מחשב לעומת טיפוסים מדויקים יותר. בגלל האופן שבו הוא עובד, אין לו Ultra Precise ויש בו עיגול של ספרות מסוימות כמו שיש עיגול בכל טיפוס מתמטי במחשב.

הוא יעיל מאוד מכיוון שיש לו פחות ספרות משמעותיות, ולכן נשתמש בו כמעט לכל פעולה עשרונית, **למעט חריג אחד: כסף**.

כאשר אנחנו מפתחים מערכות פיננסיות, לא נשתמש ב-`double` לאחסון כסף כדי לא לאבד אגורות בגלל עיגולי מסגרת.

במקרים אלו, נדרוש דיוק גבוה לחלוטין ובדרך כלל נשתמש בטיפוס `decimal`.